MPL em execução

Torre de Hanoi com três discos e sete movimentos

Henrique Motta Canho · Guilherme Bertozzo
Yago Leme · Victor Hugo Machado

```
./compilar exemplos/torre_hanoi.mpl
./executar exemplos/torre_hanoi.mplb
```

Duas funções recursivas

hanoi imprime os movimentos. quantidade calcula o total. O programa compara esse total com uma expressão aritmética.

```
A -> C
A -> B
C -> B
A -> C
B -> A
B -> C
A -> C
```

Total: 7

Tokens e árvore sintática

Linha 21: passos = 2 * 3 + 1;

Tokens com posição

```
21,3,ID,passos
21,10,ATRIBUI,=
21,12,INTEIRO,2
21,14,VEZES,*
21,16,INTEIRO,3
21,18,MAIS,+
21,20,INTEIRO,1
21,21,PONTO_VIRGULA,;
```

Multiplicação antes da soma

```
atribuicao passos
  binario +
    binario *
      literal inteiro 2
      literal inteiro 3
    literal inteiro 1
```

O parser usa um nível por precedência. A soma fica na raiz e a multiplicação no filho esquerdo.

Escopos e código intermediário

A semântica resolve o nome antes da geração

Símbolo da variável

```
escopo 8 bloco pai 7
  passos|variavel|inteiro|20

binding: v8_passos
slot no IR: v5
```

O vínculo distingue variáveis de mesmo nome em blocos diferentes. O destino exige inteiro.

Uma operação por linha

```
t38 = 2
t39 = 3
t40 = t38 * t39
t41 = 1
t42 = t40 + t41
v5 = t42
```

t40 guarda 2 * 3. t42 soma 1. A cópia final grava 7 no slot de passos.

O arquivo executável .mplb

Instruções textuais com a posição do comando original

```
21:3 PUSH [2]
21:3 STORE ["t38"]
21:3 PUSH [3]
21:3 STORE ["t39"]
21:3 LOAD ["t38"]
21:3 LOAD ["t39"]
21:3 BINARY ["*", "inteiro"]
21:3 STORE ["t40"]
```

```
21:3 PUSH [1]
21:3 STORE ["t41"]
21:3 LOAD ["t40"]
21:3 LOAD ["t41"]
21:3 BINARY ["+", "inteiro"]
21:3 STORE ["t42"]
21:3 LOAD ["t42"]
21:3 STORE ["v5"]
```

PUSH empilha um literal. LOAD lê um slot. BINARY retira dois operandos e empilha o resultado. STORE grava o topo em um slot.

Recursão na máquina virtual

Cada chamada conserva seu próprio estado

CALL cria um registro de ativação

O registro guarda locais, parâmetros, base da pilha de operandos e endereço de retorno. A VM usa um laço iterativo.

RETURN restaura o chamador

O retorno remove o registro atual e devolve o valor, quando existe. Assim hanoi pode se chamar sem sobrescrever os dados da chamada anterior.

Limite: 4096 registros. Estouro gera erro de execução.

Um separador Unicode quebrava o bytecode

Caso reproduzível da implementação assistida

Sintoma

Um texto com o caractere U+2028 compilava, mas a VM rejeitava o arquivo gerado.

Diagnóstico

A serialização conservava o caractere real. splitlines() interpretava U+2028 como outra linha e cortava a instrução PUSH.

Correção e regressão

JSON com ensure_ascii=True conserva uma instrução por linha. O teste confirma que o texto original volta intacto na saída.

Preparação para a alteração ao vivo

Material de apoio após os cinco minutos

****	Lexer, precedência do parser, tipos e operação da VM
senaose	Nova palavra reservada e encadeamento de nós se
escreva(a, b)	Lista de argumentos, tipos e uma sequência de escrita
pare	Contexto de laço na semântica e salto para o fim na IR
e / ou	Desvios condicionais para executar o lado direito só se necessário

Dez minutos para implementar, três minutos para demonstrar.

Parser manual e gerador de parser

Comparação para a defesa

Implementação deste trabalho

Funções Python reconhecem cada nível de precedência. É possível acompanhar diretamente o consumo de tokens e a montagem da árvore.

Com um gerador

Uma gramática declarativa alimentaria uma ferramenta, como ANTLR. Ainda seriam necessárias ações para montar a AST e as fases de semântica e geração.

O enunciado exige implementação manual nas entregas 1 e 2.